

HBAC: Hierarchical Budgeted Agent Control Under Token Scarcity

Vikranth Nara

<https://github.com/vikranthnara/HBAC>

Abstract

When multiple LLM agent tasks share a **global token budget**, allocators must decide per-task caps and when to stop reasoning. We present **Hierarchical Budgeted Agent Control (HBAC)**, a two-stage system: a GRPO-trained Level-1 schema allocator sets batch budgets, and a PPO-trained Level-2 stop head halts frozen ReAct rollouts [Yao et al., 2023]. We evaluate on a **V3 real pool** (LiveCodeBench + SWE Lite oracles, τ -bench, ToolBench) with Tier-A official CLEAR, ZEBRA, and Re-FORC baselines [Wan et al., 2026, Hamri and Talgam-Cohen, 2026, Zabounidis et al., 2025], plus a legacy stub micro-harness for controlled ablations.

On **oracle replay** at 40% global budget, HBAC achieves **80% pass@1** vs. **60%** for uniform and CLEAR (+20 pp; $n=300$ –500 tasks). A **type-prior heuristic ties HBAC at 80%** on oracle but starves LCB/SWE budget; **hbac_fair** (fairness reserve) trades oracle pass@1 (74.7%) for non-zero hard-task allocation. On **live** Qwen2.5-7B + DPO v2 LoRA at **floor=400**, **hbac_fair beats type-prior by +1.3 pp** (27.65% vs. 26.35%, $n=2000$; CIs overlap)—the gap is **floor-invariant** across 300–600 and driven by LiveCodeBench (3.25% vs. 0%), not oracle allocation. Legacy stub eval ($n=300$) shows pass@1 ties at generous floors (+27.6 ppvs. uniform only under tight floors). We report negative results openly: ROI-skip is reward hacking; **hard_min_frac** sweep does not close the oracle gap; SWE pass@1 remains 0% live. Controller overhead is $\sim 19 \mu$ s per allocation. Code, locked artifacts, and heuristic baselines are released.

1 Introduction

Batch agent evaluation—running SWE-bench [Jimenez et al., 2024], ToolBench [Qin et al., 2023], τ -bench [Yao et al., 2025], and related suites under a shared GPU-hour or token quota—is increasingly common. **Uniform** allocation ignores heterogeneous difficulty; **per-query economic allocators** (CLEAR [Wan et al., 2026], ZEBRA [Hamri and Talgam-Cohen, 2026]) optimize single-task emergence curves and may not respect batch structure; **per-turn methods** (TAB [Jali et al., 2026], Re-FORC [Zabounidis et al., 2025]) do not allocate across tasks.

We study HBAC, which decouples (i) a contextual bandit over allocation schemas (Level-1, GRPO [Shao et al., 2024]) from (ii) a binary stop classifier over frozen ReAct rollouts (Level-2, PPO [Schulman et al., 2017]). Our evaluation is deliberately honest: we report where a 20-line type-prior heuristic *ties* learned allocation on oracle replay, and where a modest live edge (+1–1.7 pp) emerges only under live generation on hard tasks.

Contributions. (1) HBAC architecture with formalized contextual-bandit + stop-classifier view (not a joint hierarchical POMDP). (2) Oracle H4: +20 pppass@1 vs. uniform/CLEAR; type-prior ties hbac_joint at 80%; hbac_fair 74.7% with softer hard-task allocation. (3) V3 live @ floor=400

($n=2000$): `hbac_fair` beats type-prior +1.3 pp; floor dose-response +1.0–1.7 ppat all floors 300–600. (4) Per-benchmark decomposition: aggregate win is LCB-only; τ /toolbench tied; SWE 0% all allocators. (5) Expanded baselines including Tier-A official CLEAR/ZEBRA/Re-FORC. (6) Negative results: ROI-skip reward hacking; `hard_min_frac` ablation cannot close oracle gap. (7) Reproducibility: locked artifacts and overhead measurements.

2 Related Work

Batch-level allocation. CLEAR [Wan et al., 2026] uses shadow-price bisection on surge utility; ZEBRA [Hamri and Talgam-Cohen, 2026] water-fills across utility curves. Both optimize *per-query* emergence under scarcity; neither jointly trains batch structure with agent stopping. In our oracle replay, CLEAR ties uniform (60% pass@1); on stub live eval CLEAR incurs violations and negative batch reward.

Per-turn control. TAB [Jali et al., 2026] learns per-turn budgets for math reasoning; Re-FORC [Zabounidis et al., 2025] applies Gittins-style early stopping. We implement batch-level *proxies* (BatchTAB, BatchRe-FORC) to test whether analogous caps close the gap—they do not beat uniform on oracle pass@1 in our mapping.

Agent training. ReAct [Yao et al., 2023] provides the frozen L2 action policy; DPO capability LoRA fixes tool-JSON for live eval. Allocator optimality (oracle) is decoupled from generation competence (DPO).

3 Method

Level 1: contextual bandit. Given batch $\mathcal{Q}$ and budget B_{total} , Level-1 selects a discrete allocation schema: $\{b_1, \dots, b_n\} = \pi^{(1)}(\mathcal{Q}, B_{\text{total}})$ with $\sum_i b_i \leq B_{\text{total}}$. GRPO updates schema logits; counterfactual credit ($\beta=0.2$) is a variance-reduction heuristic, not an unbiased gradient estimator.

Level 2: stop classifier. Per-task rollout stops when $\pi_{\text{stop}}^{(2)}(s_t)=1$, budget is exhausted, or success is reached. L2 does not co-train the LLM; episodes follow a fixed Markov chain conditioned on per-task budget.

Rewards. Level-2 terminal reward: $R_i^{(2)} = S_i - \lambda C_i - \gamma L_i - \delta R_i - \eta \cdot \mathbb{I}[\text{premature stop}]$. Level-1 batch reward (Variant B GRPO): $R^{(1)} = \text{pass_rate} - \lambda_v \cdot \text{violations} + \beta \cdot \text{Var}_b(\text{budget}_b)$. Optional `hbac_fair`: apply `fairness_reserve_alloc` with `hard_min_frac=0.15` so hard benchmarks receive a minimum budget fraction.

Metrics. Primary tables report raw pass@1, mean tokens, batch violation rate, and mean batch reward. Compliant utility (Appendix A) is secondary.

Controller overhead. L1 allocation latency is $\sim 19 \mu\text{s}$ per batch (~ 1 proxy control token vs. ~ 94 tokens saved per task on stub live eval).

Table 1: Oracle replay on stub curriculum batches ($n=300$, 40% global budget).

Allocator	pass@1	Mean $R^{(1)}$	Violations	Tokens/task
HBAC	80.0%	1.02	0%	—
Uniform	60.0%	0.60	0%	—
CLEAR (proxy)	60.0%	0.15	9.1%	—
ZEBRA (proxy)	60.0%	0.90	0%	47.3
SJF	40.0%	0.44	0%	48.0
Type-prior	80.0%	1.35	0%	37.0
TAB / Re-FORC proxy	60.0%	~ 0.15	0%	~ 55

4 Experimental Setup

We run two tracks with shared L1/L2 checkpoints (Variant B tight bf040; Qwen2.5-7B-Instruct + DPO v2 LoRA 20260705T014948Z_capability_v2).

Track A: stub micro-harness. ToolBench, τ -bench, mock, and SWE-bench *stubs* (deterministic trajectories). Oracle: 300 tasks / 30 batches (tight bf040). Live: 300 tasks / 50 batches; per-task floors 300–600. Stub ceilings: toolbench $\sim 100\%$, $\tau \sim 33\%$, SWE 0%.

Track B: V3 real pool. Oracle pool `data/oracles/real_eval/latest`: LiveCodeBench [Jain et al., 2024] + SWE Lite oracles, τ -bench, ToolBench ($n=300$ tasks / 30 batches for oracle matrix). Live: $n=2000$ tasks / 200 batches; floor=400 primary; floor sweep 300–600 ($n=300$ per floor). Tier-A official CLEAR, ZEBRA, Re-FORC plus heuristics (SJF, type-prior).

Baselines. Tier-B proxies (CLEAR, ZEBRA, BatchTAB, BatchRe-FORC) follow published mechanisms but are not author codebases. Tier-A official implementations follow paper algorithms; author GitHub repos were not available at submission time. Bootstrap 95% CIs use 2000 resamples.

5 Results

5.1 Oracle replay

Curriculum stub pool (Track A). Table 1 summarizes the tight heterogeneous batch matrix. HBAC +20 pp vs. uniform/CLEAR is the primary supported oracle claim. Type-prior **ties** HBAC on pass@1 with higher batch reward by zeroing SWE budget—HBAC learns a softer version without hand-coded taxonomy.

V3 real pool (Track B). Table 2 reports the real_eval oracle matrix ($n=300$).

Oracle vs. live. On oracle replay, success follows deterministic trajectories—type-prior’s zero LCB/SWE budget does not hurt pass@1 when hard tasks are unsolvable in the index. **Live generation** breaks this: Qwen2.5-7B + DPO v2 solves some LiveCodeBench problems when allocated budget. Type-prior zeros LCB live ($\rightarrow 0\%$ LCB pass@1); hbac_fair reserves 15% ($\rightarrow 3.25\%$

Table 2: V3 real-pool oracle replay ($n=300$, 40% budget).

Allocator	pass@1	Mean $R^{(1)}$
HBAC joint	80.0%	0.98
Type-prior	80.0%	1.24
HBAC fair	74.7%	0.76
Uniform / CLEAR / Tier-A official	60.0%	~ 0.60
ZEBRA (compose)	73.3%	0.74

Table 3: V3 live heuristic baselines (floor=400, $n=2000$, hbac.fair with fairness reserve).

Allocator	pass@1	95% CI
HBAC joint	27.95%	26.1–29.9%
HBAC fair	27.65%	25.8–29.6%
Type-prior	26.35%	24.5–28.2%
ZEBRA (proxy)	22.15%	20.4–24.0%
Re-FORC official	16.35%	14.8–18.0%
CLEAR official	11.60%	10.3–13.0%
Uniform / compose proxies	$\sim 11.3\%$	~ 10.0 – 12.7%
ZEBRA official	9.2%	8.0–10.5%
SJF	4.3%	3.4–5.2%

LCB). The aggregate +1.3 ppwin is a **generation-regime effect**, not oracle allocation superiority (confirmed by `hard_min_frac` sweep, §6).

5.2 Live evaluation: stub track (Track A)

Generous floor (600). HBAC, uniform, and CLEAR all reach 44.3% pass@1 (stub ceilings dominate). HBAC leads on batch reward (14.70 vs. 0.44) and token efficiency (~ 94 fewer tokens/task) with 0% violations; CLEAR has -0.27 reward and 14.3% violations.

Tight floor (400). HBAC **44.3%** vs. uniform **16.7%** (+27.6 pp). Gap emerges when uniform cannot fund multi-step tool chains; not observed at floor=600.

5.3 V3 live heuristics ($n=2000$, floor=400)

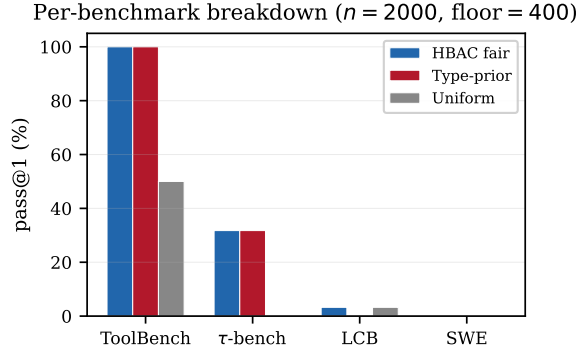
Table 3 summarizes Track B live results.¹

HBAC fair beats type-prior by **+1.3 pp**; CIs overlap—modest separation, not dominance. Figure 1a shows the gap is **LCB-only**; τ -bench tied; SWE 0% for all.

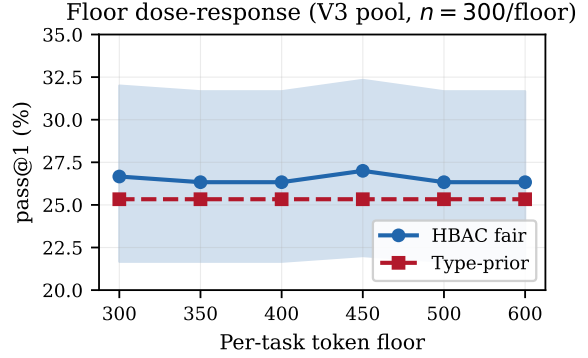
5.4 Floor dose-response (hbac.fair vs. type-prior)

HBAC fair beats type-prior at all six floors (300–600): mean gap **+1.2 pp** (range +1.0 to +1.7 pp). Type-prior is flat at $\sim 25.3\%$ across floors; separation is **floor-invariant** on the V3 pool (Figure 1b). Per-floor CIs overlap; $n=2000$ @ floor=400 confirms direction at scale but not statistical dominance.

¹Artifact filename uses legacy tag `n1000`; evaluation uses $n=2000$ tasks.



(a) Per-benchmark pass@1.



(b) Floor dose-response.

Figure 1: V3 live analysis: (a) per-benchmark breakdown explains the aggregate gap; (b) hbac_fair beats type-prior at all six floors ($n=300/\text{floor}$).

6 Ablations and Negative Results

Hypothesis tests. H4 (+20 pporacle) confirmed; H5/H6/D6/D16 falsified or neutral; H7 (KL prevents stop collapse) confirmed.

D14 ROI-skip (negative). Zeros SWE budget below floor=350 and redistributes to ToolBench—shortest-job-first *task dropping*, not learned allocation. Reported +27.7 pp@ floor=300 is an upper bound on heuristic cheating; do not deploy.

Type-prior ties HBAC (oracle). A 20-line heuristic matches HBAC pass@1 on oracle. HBAC value shifts to learning without taxonomy labels, softer fairness, and live edges where generation makes LCB partially solvable.

hard_min_frac sweep (D17). On real_eval oracle ($n=300$), hbac_fair is flat at **74.7%** for frac 0.10–0.25; type-prior fixed at **80.0%**. **Verdict:** never beats type-prior on oracle; live +1.3 ppis not explained by oracle allocation superiority.

7 Limitations

(1) V3 adds real LCB/SWE Lite oracles but SWE live pass@1 is still 0%; τ /toolbench partially stubbed. (2) $n=2000$ live gap (+1.3 pp) has overlapping CIs—directionally consistent, not statistically dominant. (3) Oracle–live disconnect: type-prior ties on oracle but loses live LCB; **hard_min_frac** tuning cannot close the oracle gap. (4) Tier-A vs. Tier-B: official baselines integrated in V3; legacy proxy results retained for stub ablations. (5) Fairness: type-prior zeros hard tasks live; hbac_fair mitigates but SWE remains unfunded. (6) DPO mixture documented but not externally audited for eval leakage. (7) L2 frozen ReAct limits joint hierarchical RL claims.

8 Conclusion

HBAC provides a reproducible harness for batch-level token allocation and early stopping under global scarcity. Oracle evaluation shows +20 pppass@1 over uniform/CLEAR; type-prior ties hbac.joint at 80%—an important honesty check. V3 live evaluation ($n=2000$) shows hbac.fair edges type-prior by +1.3 ppat floor=400, confirmed across a six-floor dose-response, with the gap localized to LiveCodeBench where type-prior starves budget under live generation. We reject reward-hacking (D14), report that `hard_min_frac` tuning cannot close the oracle gap, and scope claims away from statistical dominance or SWE SOTA. Locked artifacts support extension with honest allocation tradeoffs rather than headline pass@1 wins.

Reproducibility. Canonical artifacts: `results/canonical_artifacts.json`. Regenerate manifest: `python -m hbac.scripts.lock_canonical_artifacts`.

References

- Mahmoud Hamri and Inbal Talgam-Cohen. Zebra: Zero-shot budgeted resource allocation for llm orchestration. *arXiv preprint arXiv:2605.20485*, 2026. ICML 2026 AgenticUQ Workshop.
- Naman Jain et al. Livecodebench: Holistic and contamination free evaluation of llms for code. *arXiv preprint arXiv:2403.07974*, 2024.
- Neharika Jali, Anupam Nayak, and Gauri Joshi. Not all turns are equally hard: Adaptive thinking budgets for efficient multi-turn reasoning. *arXiv preprint arXiv:2604.05164*, 2026.
- Carlos E. Jimenez et al. Swe-bench: Can language models resolve real-world github issues? In *ICLR*, 2024.
- Yujia Qin et al. Toolllm: Facilitating llms to master 16000+ real-world apis. *arXiv preprint arXiv:2307.16789*, 2023.
- John Schulman et al. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Zhihong Shao et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Xu Wan et al. The shadow price of reasoning: Economic perspective on optimal budget allocation for llms. *arXiv preprint arXiv:2606.03092*, 2026. ICML 2026.
- Shunyu Yao et al. React: Synergizing reasoning and acting in language models. In *ICLR*, 2023.
- Shunyu Yao et al. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. In *ICLR*, 2025.
- Renos Zabounidis et al. Re-forc: Adaptive reward prediction for efficient chain-of-thought reasoning. *arXiv preprint arXiv:2511.02130*, 2025. NeurIPS 2025 Efficient Reasoning Workshop.

A Compliant Utility

Secondary metric: $U = R \cdot (1 - \text{violation_rate}) - 0.5 \cdot \text{parse_failures}$. Primary tables use raw pass@1, tokens, and violations.

B DPO v2 Data Mixture

600 `wrong_tool` rejection pairs from training oracles; 3-epoch SFT warmstart + 3-epoch DPO; 100% tool-name match on format eval. Checkpoint: 20260705T014948Z_capability_v2.

C Artifact Index

Claim	File
Oracle H4 (+20 pp)	<code>compose_tight_bf040_seed47.json</code>
V3 live $n=2000$	<code>compose_live_v3_heuristics_floor400_n1000_dpo_v2.json</code>
V3 fair floor sweep	<code>fair_floor_sweep_analysis.json</code>
V3 oracle matrix	<code>v3_real_oracle_matrix.json</code>
<code>hard_min_frac</code> ablation	<code>hard_min_frac_oracle_sweep.json</code>